# CSE 270 Sports Analytics

## Predicting with distributions

Habet Madoyan

# Predicting with distributions

# Predicting with distributions

Suppose we have two variables with normal distribution,

- Home team score
- Away team score

$$H \sim N(\mu_H, \sigma_H^2)$$

$$A \sim N(\mu_A, \sigma_A^2)$$

Then the difference between the home and away teams scores will also have normal distribution with the following parameters (assuming this two are independent)

$$\mu_d = \mu_H - m_A$$
$$\sigma_D^2 = \sigma_H^2 + \sigma_A^2$$

Thus the difference will have a normal distribution with the following parameters

$$D \sim N(\mu_H - \mu_A, \sigma_H^2 + \sigma_A^2)$$

# Predicting with distributions

- In NBA (National Basketball Association) points scored per game by home and away teams follow normal distribution.
- Install package SportsAnalytics270 from github.
- The package contains data for Sports analytics.

```
library(devtools)
install_github("HABET/CSE270")
library(SportsAnalytics270)
```

# Predicting with distributions

The list of necessary packages

```
library(SportsAnalytics270)
library(dplyr)
library(ggplot2)
library(reshape2)
library(zoo)
library(VGAM)
```

# Predicting with distributions

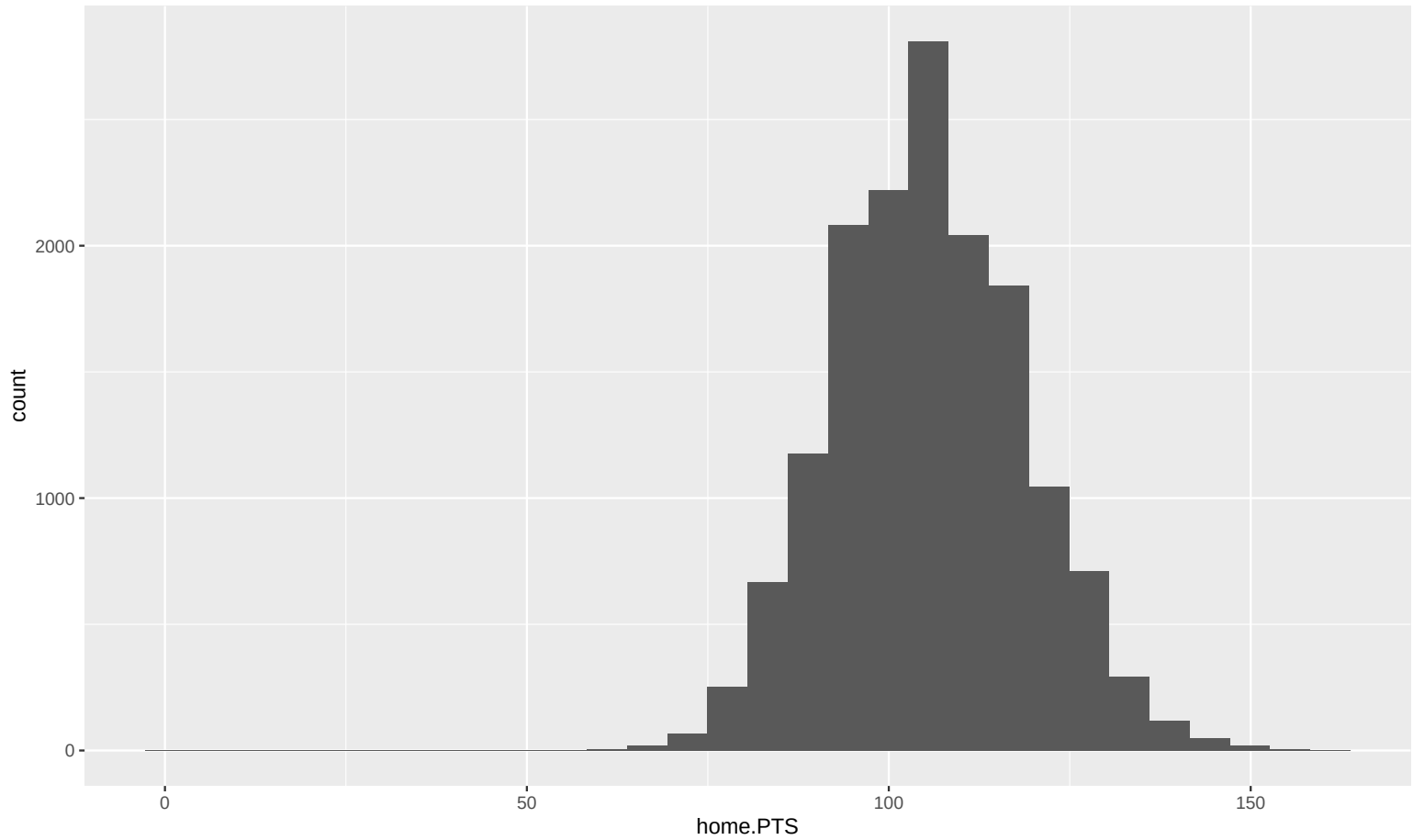
Dataframe nba2009_2021 contains NBA regular season games from 2009 to 2021-2022 season

```
options(width = 65)
data("nba2009_2021")
head(nba2009_2021, 2)
##   SEASON_ID  GAME_DATE home.TEAM_ABBREVIATION
## 1      2009 2009-10-27                    CLE
## 2      2009 2009-10-27                    DAL
##        home.TEAM_NAME home.PTS away.TEAM_ABBREVIATION
## 1 Cleveland Cavaliers       89                    BOS
## 2    Dallas Mavericks       91                    WAS
##       away.TEAM_NAME away.PTS home.WL
## 1     Boston Celtics       95       L
## 2 Washington Wizards      102       L
```

# Predicting with distributions

Plot the histogram for home points

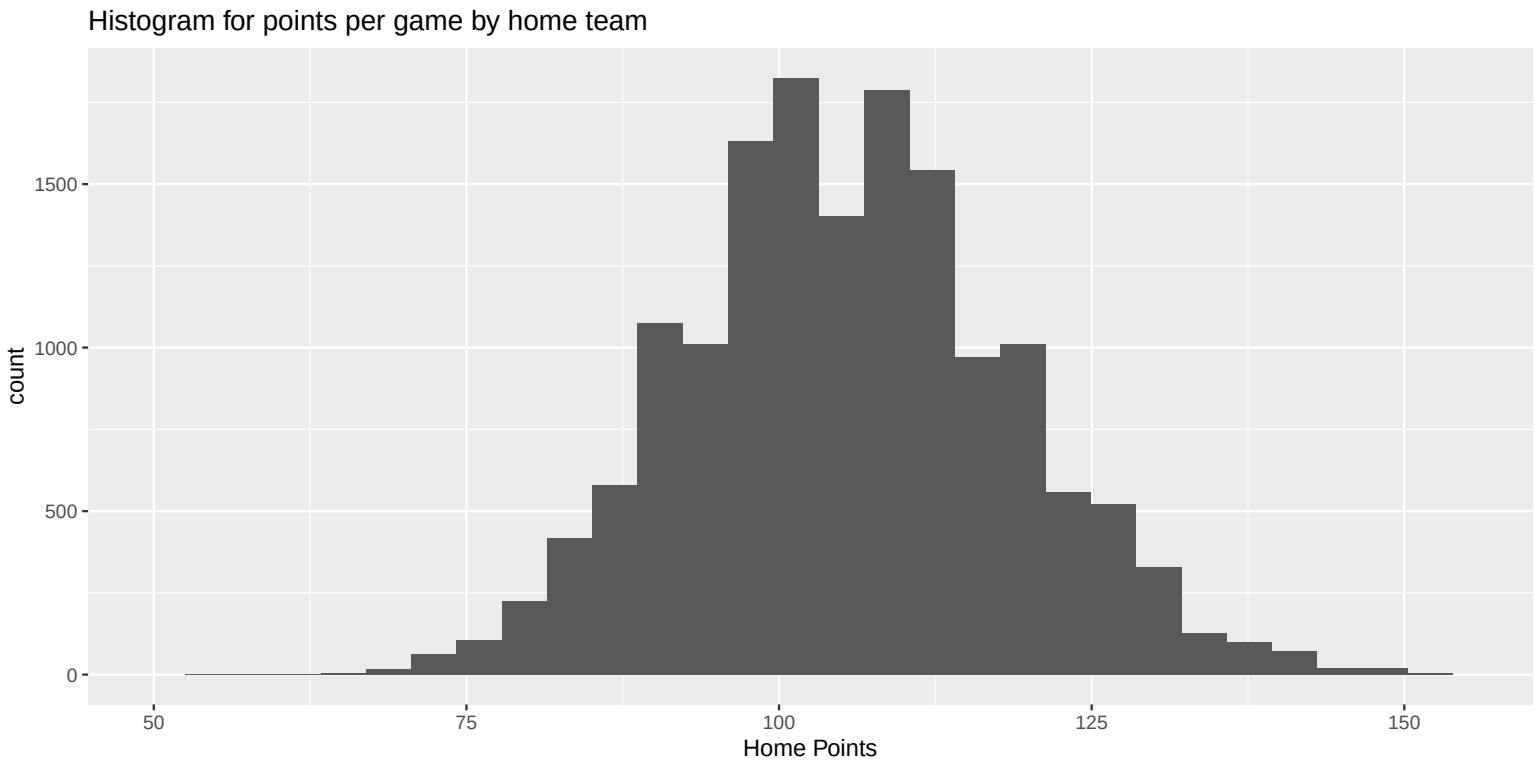
```
ggplot(nba2009_2021, aes(x=home.PTS))+geom_histogram()
```



# Predicting with distributions

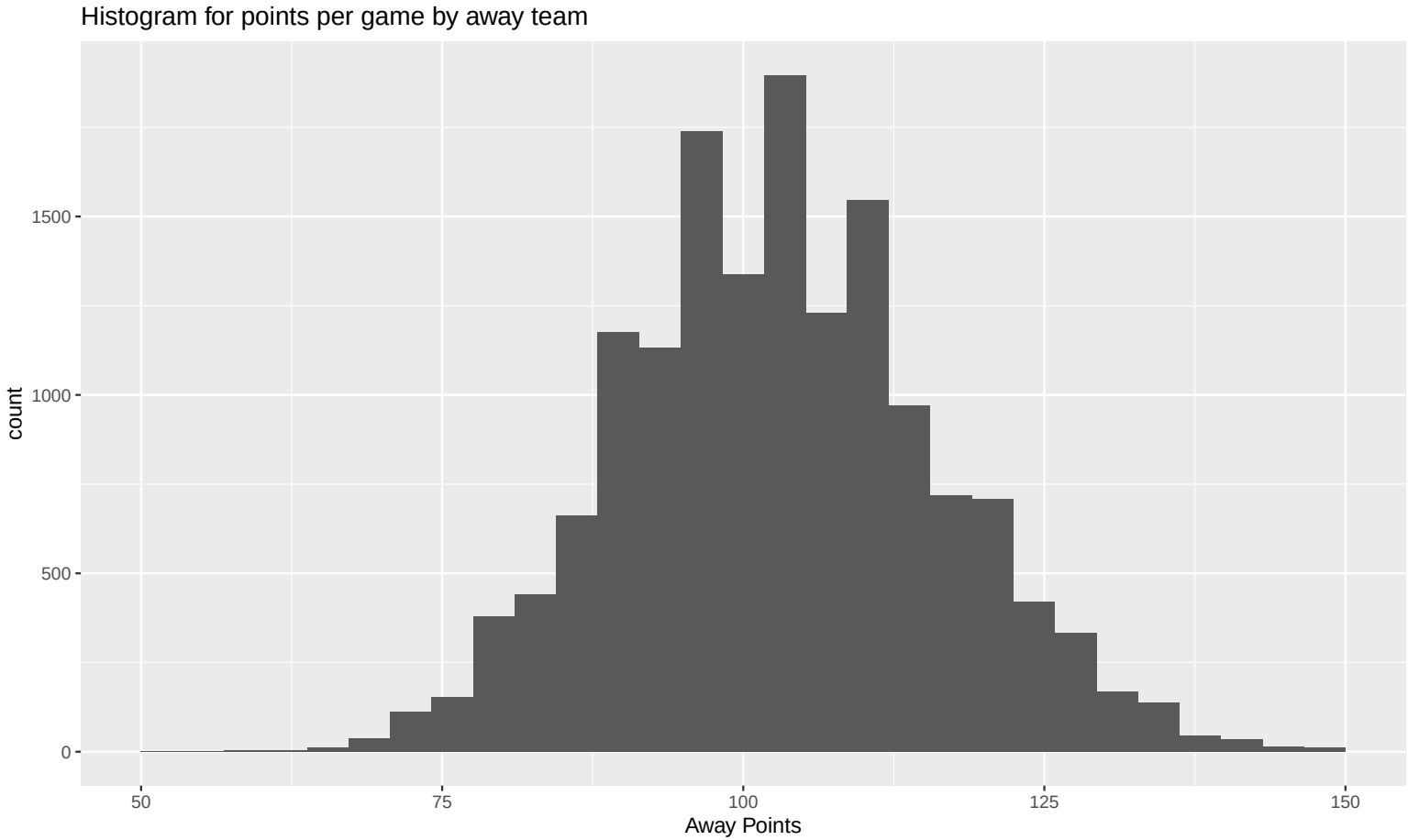
Exclude Boston-Celtics game

```
ggplot(nba2009_2021, aes(x=home.PTS)) + geom_histogram() + xlim(c(50,155))+
  labs(x="Home Points", title="Histogram for points per game by home team")
```



# Predicting with distributions

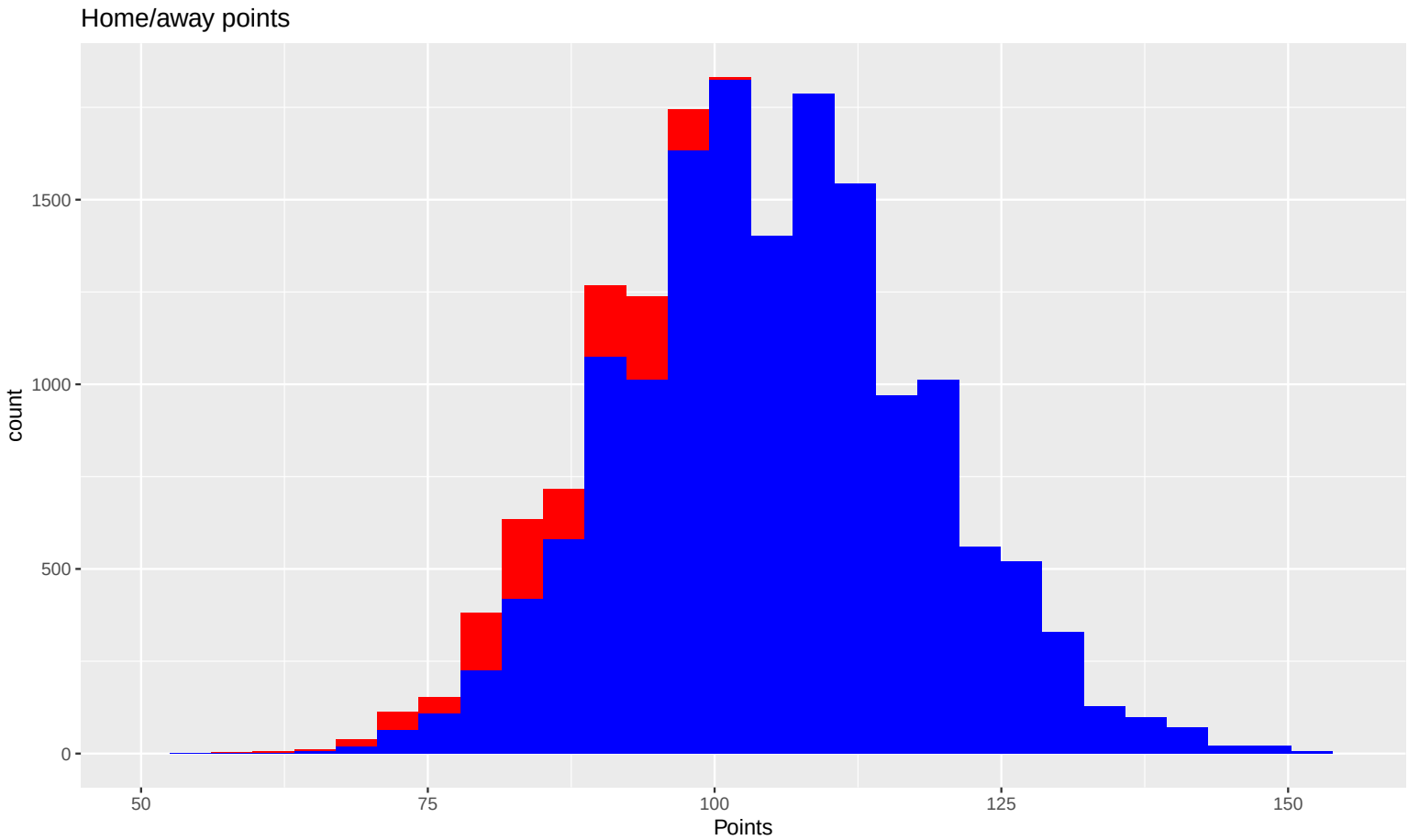
```
ggplot(nba2009_2021, aes(x=away.PTS)) + geom_histogram() + xlim(c(50,150)) +
  labs(x="Away Points", title="Histogram for points per game by away team")
```



# Predicting with distributions

Histogram of home and away points together

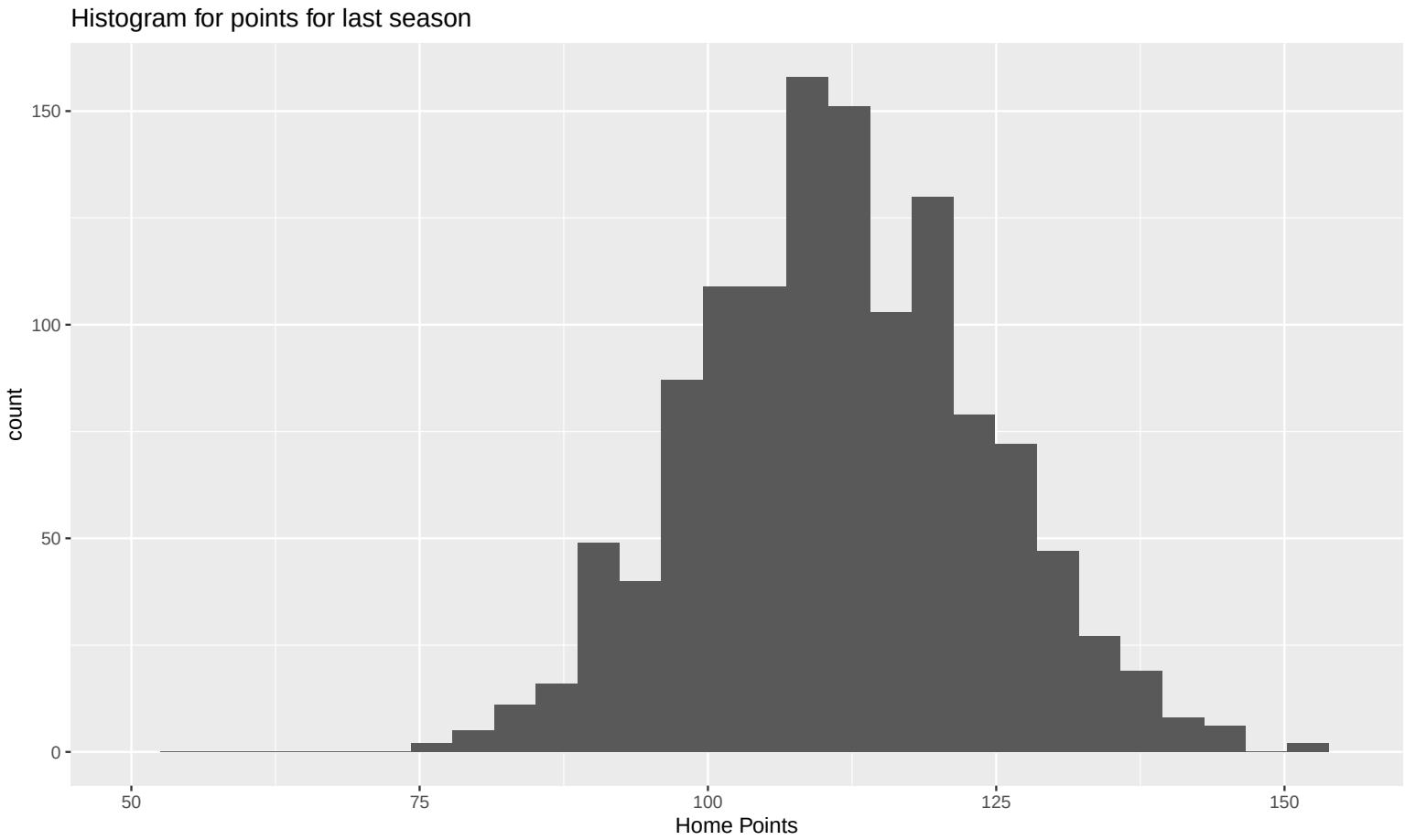
```
ggplot(nba2009_2021) + geom_histogram(aes(x=away.PTS), fill='red') +
  geom_histogram(aes(x=home.PTS),fill='blue') +
  xlim(c(50,155)) + labs(x="Points", title="Home/away points")
```



# Predicting with distributions

Home points for last season only

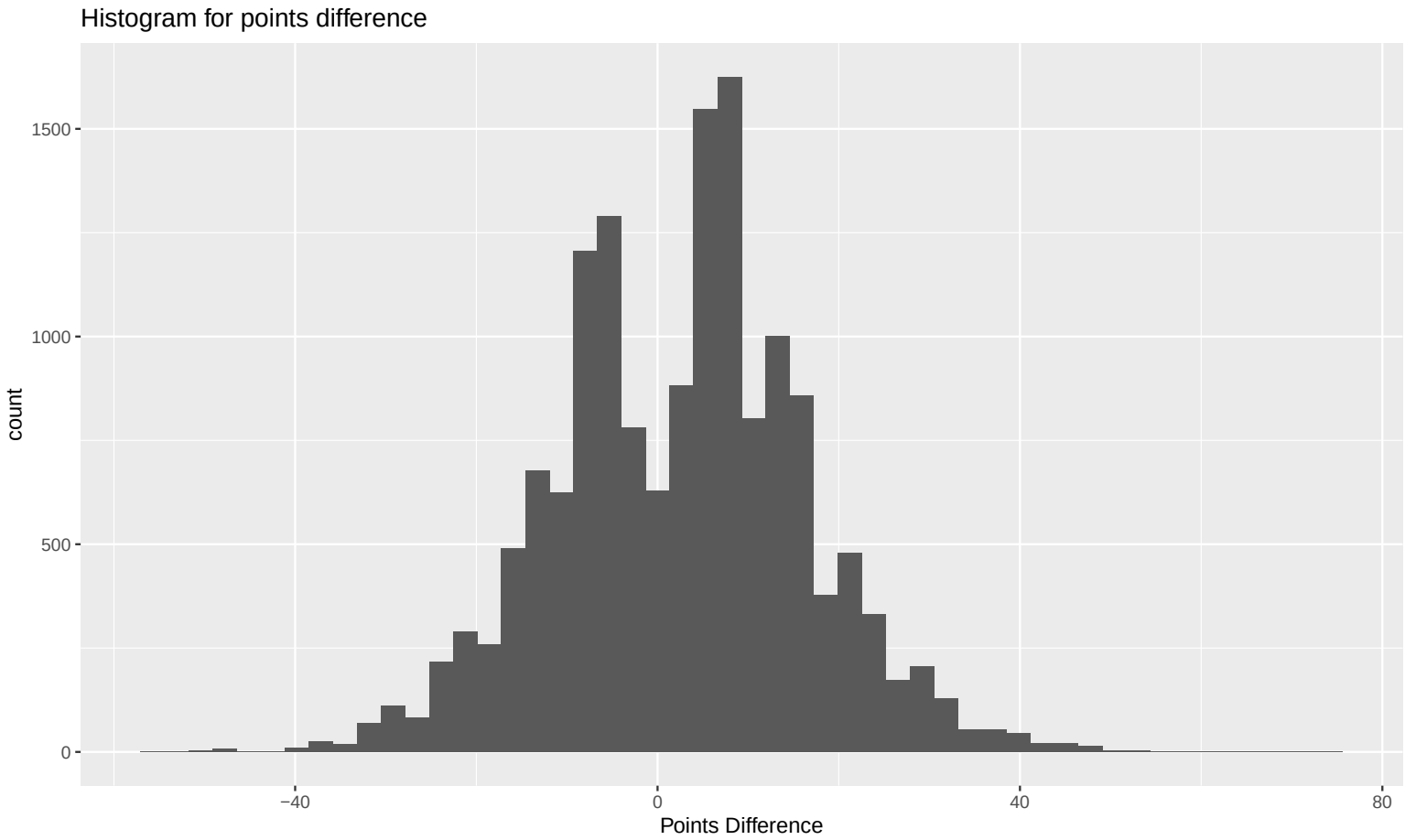
```
nba2009_2021 %>% filter(SEASON_ID=='2021') %>%
ggplot(aes(x = home.PTS)) + geom_histogram() + xlim(c(50,155))+
  labs(x="Home Points", title="Histogram for points for last season")
```



# Predicting with distributions

What about Points difference ?

```
nba2009_2021$Diff = nba2009_2021$home.PTS-nba2009_2021$away.PTS
ggplot(nba2009_2021, aes(x=Diff)) + geom_histogram(bins=50) +
  labs(x="Points Difference", title="Histogram for points difference")
```



# Predicting with distributions

Test hypothesis of Normality:

$$H_0 : \textit{sample came from normal distribution}$$
$$H_1 : \textit{sample did not come from normal distribution}$$

Find the p-value of the test: if $\alpha$ is more than 0.05 then it has a normal distribution failed to reject $H_0$

# Predicting with distributions

```
nba2009_2021 %>%
  filter(SEASON_ID=='2021') %>%
  group_by(home.TEAM_ABBREVIATION) %>%
  summarise(p_value = shapiro.test(home.PTS)$p.value)%>%
    arrange(p_value)
## # A tibble: 30 x 2
##    home.TEAM_ABBREVIATION p_value
##    <chr>                    <dbl>
##  1 ATL                     0.0449
##  2 MIL                     0.0507
##  3 NYK                     0.0784
##  4 CLE                     0.105
##  5 GSW                     0.111
##  6 ORL                     0.116
##  7 CHI                     0.129
##  8 PHX                     0.223
##  9 SAS                     0.270
## 10 IND                     0.297
## # ... with 20 more rows
```

# Predicting with distributions

Away points: Normal distribution with $\alpha = 0.05$

```
nba2009_2021 %>%
  filter(SEASON_ID=='2021') %>%
  group_by(away.TEAM_ABBREVIATION) %>%
  summarise(p_value = shapiro.test(away.PTS)$p.value)%>%
  arrange(p_value)
## # A tibble: 30 x 2
##    away.TEAM_ABBREVIATION p_value
##    <chr>                    <dbl>
##  1 MIN                     0.0429
##  2 LAC                     0.0545
##  3 SAS                     0.157
##  4 CHI                     0.188
##  5 OKC                     0.193
##  6 CHA                     0.214
##  7 ATL                     0.280
##  8 TOR                     0.285
##  9 NYK                     0.287
## 10 POR                     0.329
## # ... with 20 more rows
```

# Predicting with distributions

Suppose we have two variables with normal distribution,

- Home team score
- Away team score

$$H \sim N(\mu_H, \sigma_H^2)$$
$$A \sim N(\mu_A, \sigma_A^2)$$

Then the difference between the home and away teams scores will also have normal distribution with the following parameters (assuming this two are independent)

$$\mu_d = \mu_H - m_A$$
$$\sigma_D^2 = \sigma_H^2 + \sigma_A^2$$

Thus the difference will have a normal distribution with the following parameters

$$D \sim N(\mu_H - \mu_A, \sigma_H^2 + \sigma_A^2)$$

# Predicting with distributions

Create dataframes with means and standard deviations for each team, for the last season

```
home = nba2009_2021 %>%
        filter(SEASON_ID=='2021') %>%
        group_by(home.TEAM_ABBREVIATION) %>%
        summarise(mean = mean(home.PTS),
                  var  = var(home.PTS))

away = nba2009_2021 %>%
        filter(SEASON_ID=='2021') %>%
        group_by(away.TEAM_ABBREVIATION) %>%
        summarise(mean = mean(away.PTS),
                  var  = var(away.PTS))
```

# Predicting with distributions

Dallas Mavericks playing @ Washington Wizards Wizards are the home team, Mavericks are the away team




@



# Predicting with distributions

Calculate the mean and standard deviation for the points difference between these two teams

$$s_D = \sqrt{\sigma_H^2 + \sigma_A^2}$$

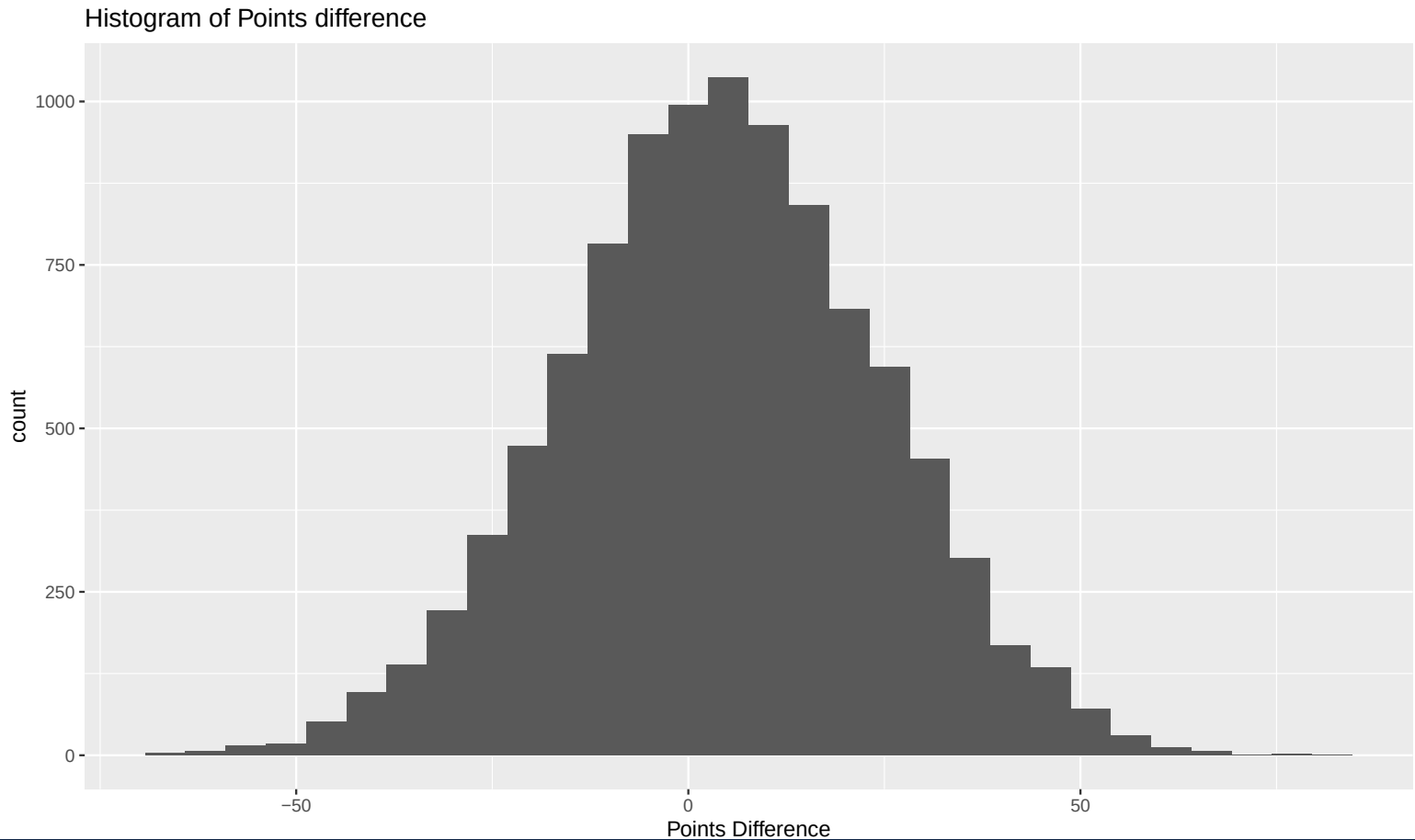
```
(mean = home$mean[home$home.TEAM_ABBREVIATION=="WAS"]-
  away$mean[away$away.TEAM_ABBREVIATION=="DAL"])
## [1] 4.219512

(sd = sqrt(home$var[home$home.TEAM_ABBREVIATION=="WAS"]+
  away$var[away$away.TEAM_ABBREVIATION=="DAL"]))
## [1] 19.89
```

# Predicting with distributions

Simulate 10,000 games and plot the histogram. Can you see the home team advantage ?

```
set.seed(1)
diff = rnorm(10000, mean, sd)
ggplot()+geom_histogram(aes(x=diff))+
  labs(x="Points Difference", title="Histogram of Points difference")
```



# Predicting with distributions

Now let's calculate the probabilities of home and away teams to win

- Away team to win $P(x < 0)$
- Home team to win $P(x > 0)$

# Predicting with distributions

### Probability of away team to win

```
pnorm(0, mean, sd, lower.tail = T)
## [1] 0.415998
```

# Predicting with distributions

### Probability of home team to win

```
pnorm(0, mean, sd, lower.tail = F)
## [1] 0.584002
```

# Predicting with distributions

**Probability of home team to win with more than 15 points difference**

Home team will win with spread of 15 points

```
pnorm(15, mean, sd, lower.tail = F)
## [1] 0.2939074
```

# Predicting with distributions

```
home_away = list(home,away)
probs = function(data, home, away, spread=0){
  home_df = data[[1]]
  away_df = data[[2]]
  mean_home = home_df$mean[home_df$home.TEAM_ABBREVIATION==home]
  mean_away = away_df$mean[away_df$away.TEAM_ABBREVIATION==away]
  mean_diff = mean_home-mean_away
  var_home = home_df$var[home_df$home.TEAM_ABBREVIATION==home]
  var_away = away_df$var[home_df$home.TEAM_ABBREVIATION==away]
  sd_diff = sqrt(var_home+var_away)
  df = data.frame(pnorm(spread, mean_diff, sd_diff, lower.tail = F),
                  pnorm(spread, mean_diff, sd_diff))
  colnames(df) = c(home, away)
  return(df)
}
```

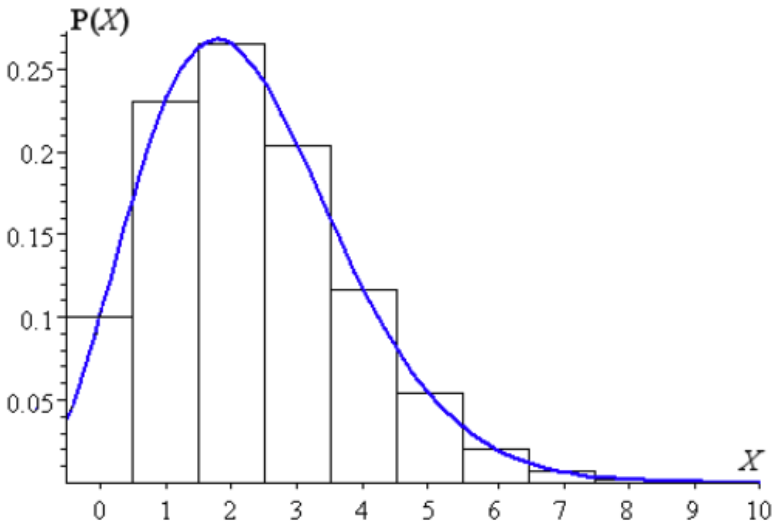
# Predicting with distributions

```
probs(home_away, home="WAS", away="DAL", 0)
##         WAS      DAL
## 1 0.584002 0.415998
```

# Predicting football games results

# Predicting football games results

Let's recall what is Poisson distribution about




$$\lambda = \frac{\sum_{i=1}^{n} x_i}{n}$$

$$f(x;\lambda) = \frac{\lambda^x e^{-\lambda}}{x!}$$

# Predicting football games results

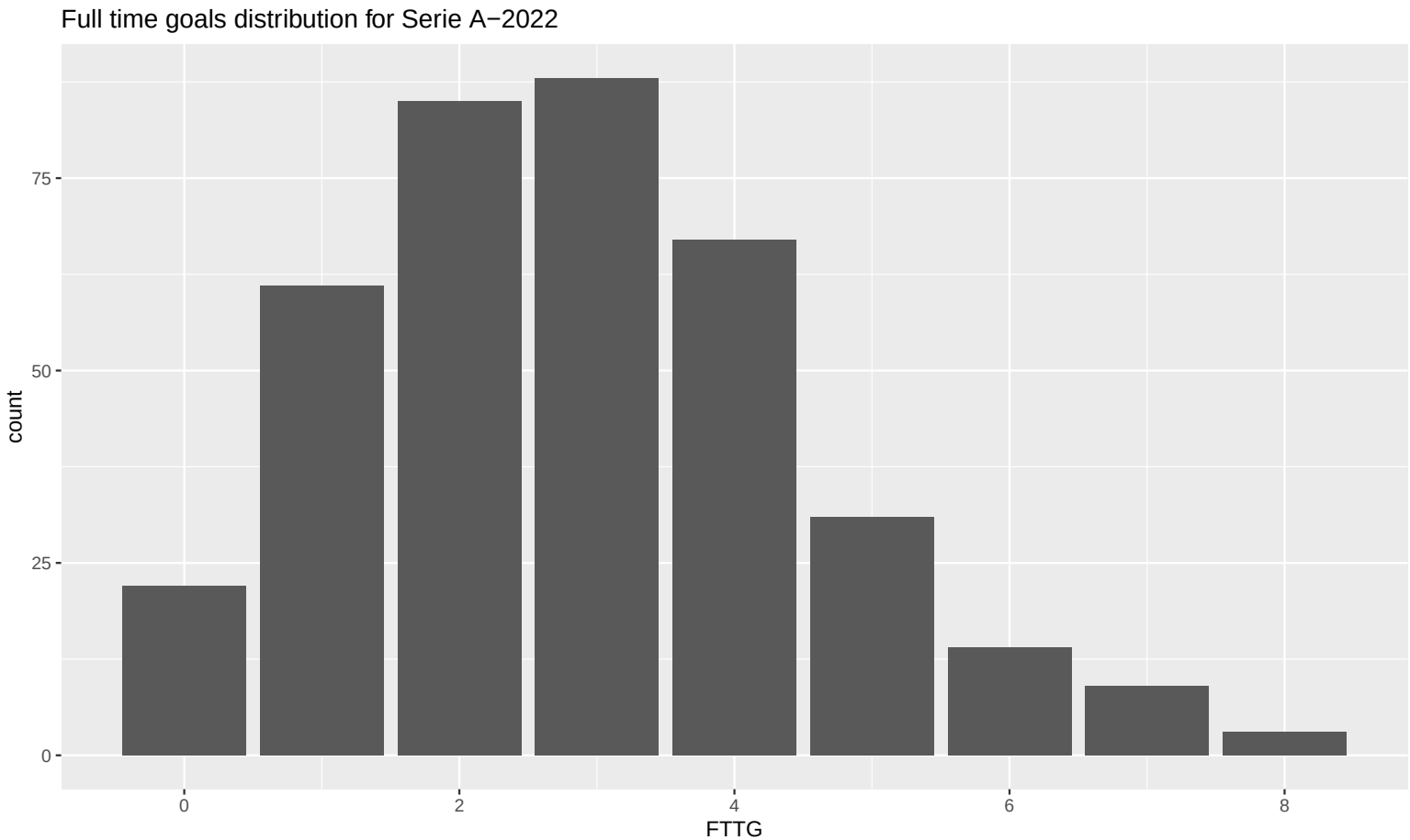
## Dataset

```
data(f_data_sm)
head(f_data_sm)
##   SEASON COUNTRY         LEAGUE       DATE    HOMETEAM
## 1   1994 England Premier League 1993-08-14     Arsenal
## 2   1994 England Premier League 1993-08-14 Aston Villa
## 3   1994 England Premier League 1993-08-14     Chelsea
## 4   1994 England Premier League 1993-08-14   Liverpool
## 5   1994 England Premier League 1993-08-14    Man City
## 6   1994 England Premier League 1993-08-14   Newcastle
##          AWAYTEAM FTSC FTR FTHG FTAG FTTG  H  D  A
## 1        Coventry  0-3   A    0    3    3 NA NA NA
## 2             QPR  4-1   H    4    1    5 NA NA NA
## 3       Blackburn  1-2   A    1    2    3 NA NA NA
## 4 Sheffield Weds  2-0   H    2    0    2 NA NA NA
## 5           Leeds  1-1   D    1    1    2 NA NA NA
## 6      Tottenham  0-1   A    0    1    1 NA NA NA
```

# Predicting football games results

## Full time total goals distribution for Serie A, Season 2021-2022

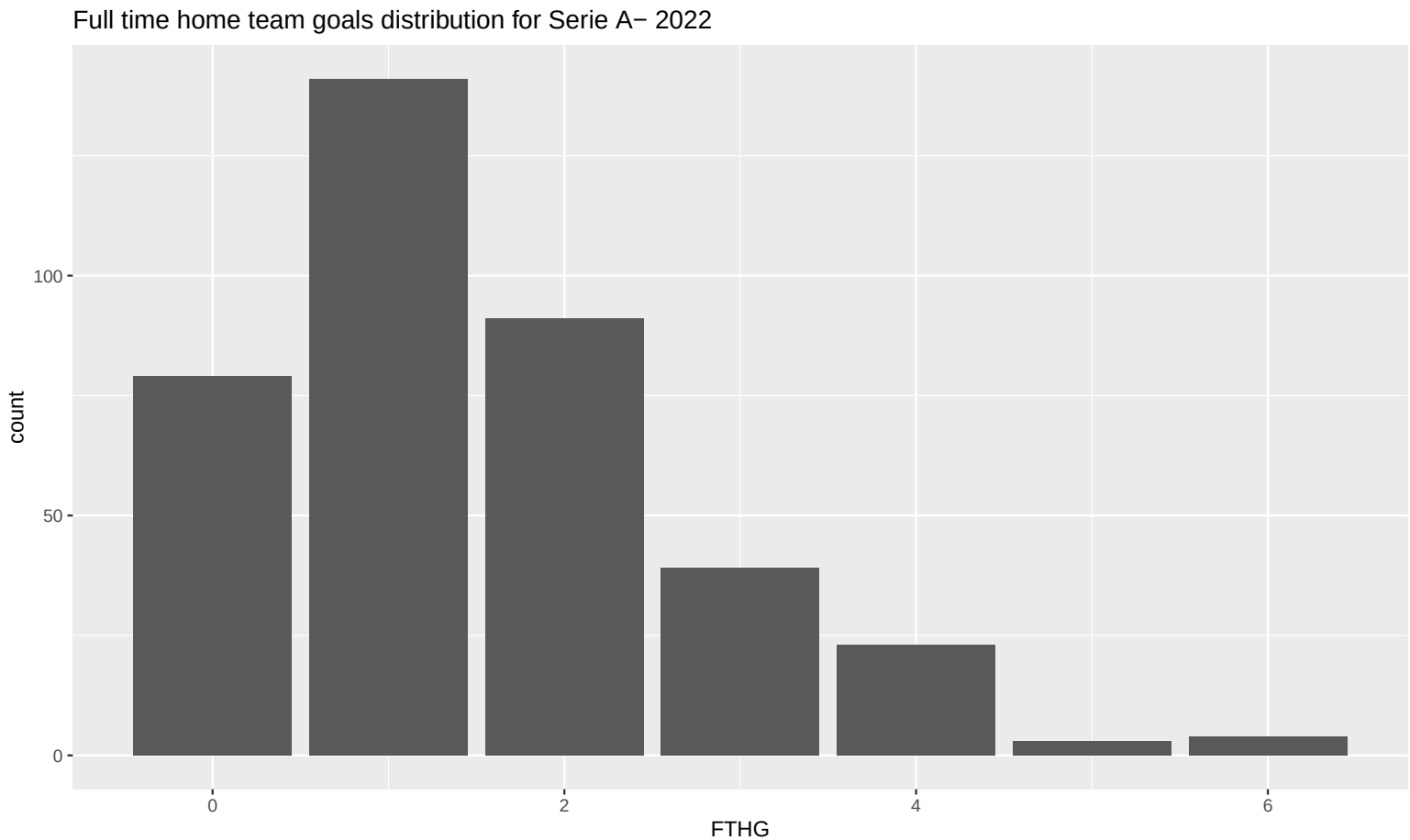
```
f_data_sm %>%
  filter(COUNTRY=="Italy", SEASON=="2022") %>%
ggplot(aes(x=FTTG))+geom_bar()+
ggtitle("Full time goals distribution for Serie A-2022")
```



# Predicting football games results

## Distribution of home team full time goals

```
f_data_sm %>%
  filter(COUNTRY=="Italy", SEASON=="2022") %>%
ggplot(aes(x=FTHG))+geom_bar()+
ggtitle("Full time home team goals distribution for Serie A- 2022")
```



# Predicting football games results

Does the total number of goals scored follows Poisson distribution?

First find the $\lambda$ (average number of goals per game)

```
f_data_sm %>%
  filter(COUNTRY=="Italy", SEASON=='2022') %>%
  summarise(mean(FTTG))
##   mean(FTTG)
## 1   2.865789
```

# Predicting football games results

We can test the hypothesis, whether the goal's distribution is Poisson: Our hypothesis is the following:

$$H_0 : \textit{The variables have Poisson distribution}$$

$$H_1 : \textit{The variables do not have Poisson distribution}$$

We will test the hypothesis with Chi-square test: thus comparing empirical probability distribution (from the data) and hypothethical Poisson distribution

# Predicting football games results

## Goal frequencies

```
(seriea = f_data_sm %>%
          filter(COUNTRY=="Italy", SEASON=="2022") %>%
          group_by(FTHG) %>%
          summarise(count = n()) %>%
          as.data.frame())
##   FTHG count
## 1    0    79
## 2    1   141
## 3    2    91
## 4    3    39
## 5    4    23
## 6    5     3
## 7    6     4
```

# Predicting football games results

We will use chi-square test

First create a vector with observed relative frequencies

```
(rel_fr=seriea$count/sum(seriea$count))
## [1] 0.207894737 0.371052632 0.239473684 0.102631579 0.060526316
## [6] 0.007894737 0.010526316
```

```
(h_goals_mean_italy <- (f_data_sm %>%
                         filter(COUNTRY=="Italy", SEASON=="2022") %>%
                           summarize(mean(FTHG)))$`mean(FTHG)`)
## [1] 1.502632
```

Then create expected relative frequencies (probabilities), under the assumption that the variable follows Poisson distribution with $\lambda$=1.5026316

```
(probs=dpois(c(0:6), lambda=h_goals_mean_italy))
## [1] 0.222543747 0.334401263 0.251240949 0.125840861 0.047273113
## [6] 0.014206814 0.003557935
```

# Predicting football games results

Is there a significant difference between these two ?

```
chisq.test(rel_fr, p=probs, rescale.p = T, simulate.p.value = TRUE)
##
##  Chi-squared test for given probabilities with simulated
##  p-value (based on 2000 replicates)
##
## data:  rel_fr
## X-squared = 0.029952, df = NA, p-value = 1
```

Large p-value talks in favor for NULL hypothesis, thus we have a Poisson distribution

# Predicting football games results

Lazio (Home team) vs. Fiorentina (Away team)




VS



# Predicting football games results

- We assume that probabilities for Lazio and Fiorentina scoring are independent from each other.
- Thus: $P(L = 0 \cap F = 0) = P(L = 0) * P(F = 0)...$
- If you want to find the probability of the draw then, $P(L = 0 \cap F = 0 \cup L = 1 \cap F = 1 \cup L = 2 \cap F = 2)$
- As $\{L = 0 \cap F = 0 \cup L = 1 \cap F = 1 \cup L = 2 \cap F = 2\}$ events are mutually exclusive, the probability of the draw can be calculated as.

$$P(L = 0 \cap F = 0) + P(L = 1 \cap F = 1) + P(L = 2 \cap F = 2)... = P(L = 0) * P(F = 0) + P(L = 1) * P(F = 1)...$$

# Predicting football games results

Means ( $\lambda$ parameters) for home and away teams

```
home_means = f_data_sm %>%
                filter(COUNTRY=="Italy", SEASON=="2022") %>%
                group_by(HOMETEAM) %>%
                summarise(mean = mean(FTHG))

away_means = f_data_sm %>%
                filter(COUNTRY=="Italy", SEASON=="2022") %>%
                group_by(AWAYTEAM) %>%
                summarise(mean = mean(FTAG))
```

# Predicting football games results

For Lazio as a home team and for Fiorentina as an away team

```
(lazio = home_means$mean[home_means$HOMETEAM=="Lazio"])
## [1] 2.210526

(fiorentina = away_means$mean[away_means$AWAYTEAM=="Fiorentina"])
## [1] 1.105263
```

# Predicting football games results

Generate vector of probabilities for number of goals scored for both teams.

Range from 0 to 9

```
options(scipen=999)
goal_probs_lazio=dpois(c(0:9), lambda=lazio)
goal_probs_fior=dpois(c(0:9), lambda=fiorentina)
```

```
goal_probs_lazio
##  [1] 0.1096429265 0.2423685744 0.2678810559 0.1973860412
##  [5] 0.1090817596 0.0482256200 0.0177673337 0.0056107370
##  [9] 0.0015503352 0.0003807841
```

```
goal_probs_fior
##  [1] 0.331123732951 0.365978862735 0.202251476775 0.074513701970
##  [5] 0.020589312386 0.004551321685 0.000838401363 0.000132379163
##  [9] 0.000018289226 0.000002246045
```

# Predicting football games results

Multiplying two vectors of probabilities we will get the matrix of probabilities for a given result

```
options(width = 300)
a=goal_probs_lazio %*% t(goal_probs_fior)
a=round(a, digits=4)
print(a)
##          [,1]   [,2]   [,3]   [,4]   [,5]   [,6]   [,7] [,8] [,9] [,10]
##  [1,] 0.0363 0.0401 0.0222 0.0082 0.0023 0.0005 0.0001    0    0     0
##  [2,] 0.0803 0.0887 0.0490 0.0181 0.0050 0.0011 0.0002    0    0     0
##  [3,] 0.0887 0.0980 0.0542 0.0200 0.0055 0.0012 0.0002    0    0     0
##  [4,] 0.0654 0.0722 0.0399 0.0147 0.0041 0.0009 0.0002    0    0     0
##  [5,] 0.0361 0.0399 0.0221 0.0081 0.0022 0.0005 0.0001    0    0     0
##  [6,] 0.0160 0.0176 0.0098 0.0036 0.0010 0.0002 0.0000    0    0     0
##  [7,] 0.0059 0.0065 0.0036 0.0013 0.0004 0.0001 0.0000    0    0     0
##  [8,] 0.0019 0.0021 0.0011 0.0004 0.0001 0.0000 0.0000    0    0     0
##  [9,] 0.0005 0.0006 0.0003 0.0001 0.0000 0.0000 0.0000    0    0     0
## [10,] 0.0001 0.0001 0.0001 0.0000 0.0000 0.0000 0.0000    0    0     0
```

# Predicting football games results

The probability that the game will end 0-0

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | **0.0363** | 0.0401 | 0.0222 | 0.0082 | 0.0023 | 0.0005 | 0.0001 | 0 | 0 | 0 |
| 1 | 0.0803 | 0.0887 | 0.0490 | 0.0181 | 0.0050 | 0.0011 | 0.0002 | 0 | 0 | 0 |
| 2 | 0.0887 | 0.0980 | 0.0542 | 0.0200 | 0.0055 | 0.0012 | 0.0002 | 0 | 0 | 0 |
| 3 | 0.0654 | 0.0722 | 0.0399 | 0.0147 | 0.0041 | 0.0009 | 0.0002 | 0 | 0 | 0 |
| 4 | 0.0361 | 0.0399 | 0.0221 | 0.0081 | 0.0022 | 0.0005 | 0.0001 | 0 | 0 | 0 |
| 5 | 0.016 | 0.0176 | 0.0098 | 0.0036 | 0.0010 | 0.0002 | 0.0000 | 0 | 0 | 0 |
| 6 | 0.0059 | 0.0065 | 0.0036 | 0.0013 | 0.0004 | 0.0001 | 0.0000 | 0 | 0 | 0 |
| 7 | 0.0019 | 0.0021 | 0.0011 | 0.0004 | 0.0001 | 0.0000 | 0.0000 | 0 | 0 | 0 |
| 8 | 0.0005 | 0.0006 | 0.0003 | 0.0001 | 0.0000 | 0.0000 | 0.0000 | 0 | 0 | 0 |
| 9 | 0.0001 | 0.0001 | 0.0001 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0 | 0 | 0 |

# Predicting football games results

3 1 Lazio winning

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0.0363 | **0.0401** | 0.0222 | 0.0082 | 0.0023 | 0.0005 | 0.0001 | 0 | 0 | 0 |
| 1 | 0.0803 | **0.0887** | 0.049 | 0.0181 | 0.005 | 0.0011 | 0.0002 | 0 | 0 | 0 |
| 2 | 0.0887 | **0.098** | 0.0542 | 0.02 | 0.0055 | 0.0012 | 0.0002 | 0 | 0 | 0 |
| 3 | **0.0654** | **0.0722** | **0.0399** | **0.0147** | **0.0041** | **0.0009** | **0.0002** | **0** | **0** | **0** |
| 4 | 0.0361 | **0.0399** | 0.0221 | 0.0081 | 0.0022 | 0.0005 | 0.0001 | 0 | 0 | 0 |
| 5 | 0.016 | **0.0176** | 0.0098 | 0.0036 | 0.001 | 0.0002 | 0 | 0 | 0 | 0 |
| 6 | 0.0059 | **0.0065** | 0.0036 | 0.0013 | 0.0004 | 0.0001 | 0 | 0 | 0 | 0 |
| 7 | 0.0019 | **0.0021** | 0.0011 | 0.0004 | 0.0001 | 0 | 0 | 0 | 0 | 0 |
| 8 | 0.0005 | **0.0006** | 0.0003 | 0.0001 | 0 | 0 | 0 | 0 | 0 | 0 |
| 9 | 0.0001 | **0.0001** | 0.0001 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

# Predicting football games results

Probability of draw: sum all elements on the diagonal

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | **0.0363** | 0.0401 | 0.0222 | 0.0082 | 0.0023 | 0.0005 | 0.0001 | 0 | 0 | 0 |
| 1 | 0.0803 | **0.0887** | 0.049 | 0.0181 | 0.005 | 0.0011 | 0.0002 | 0 | 0 | 0 |
| 2 | 0.0887 | 0.098 | **0.0542** | 0.02 | 0.0055 | 0.0012 | 0.0002 | 0 | 0 | 0 |
| 3 | 0.0654 | 0.0722 | 0.0399 | **0.0147** | 0.0041 | 0.0009 | 0.0002 | 0 | 0 | 0 |
| 4 | 0.0361 | 0.0399 | 0.0221 | 0.0081 | **0.0022** | 0.0005 | 0.0001 | 0 | 0 | 0 |
| 5 | 0.016 | 0.0176 | 0.0098 | 0.0036 | 0.001 | **0.0002** | 0 | 0 | 0 | 0 |
| 6 | 0.0059 | 0.0065 | 0.0036 | 0.0013 | 0.0004 | 0.0001 | **0** | 0 | 0 | 0 |
| 7 | 0.0019 | 0.0021 | 0.0011 | 0.0004 | 0.0001 | 0 | 0 | **0** | 0 | 0 |
| 8 | 0.0005 | 0.0006 | 0.0003 | 0.0001 | 0 | 0 | 0 | 0 | **0** | 0 |
| 9 | 0.0001 | 0.0001 | 0.0001 | 0 | 0 | 0 | 0 | 0 | 0 | **0** |

# Predicting football games results

Probability of Lazio winning: Sum of all the probabilities in the lower triangle, without diagonal

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0.0363 | 0.0401 | 0.0222 | 0.0082 | 0.0023 | 0.0005 | 0.0001 | 0 | 0 | 0 |
| 1 | **0.0803** | 0.0887 | 0.049 | 0.0181 | 0.005 | 0.0011 | 0.0002 | 0 | 0 | 0 |
| 2 | **0.0887** | **0.098** | 0.0542 | 0.02 | 0.0055 | 0.0012 | 0.0002 | 0 | 0 | 0 |
| 3 | **0.0654** | **0.0722** | **0.0399** | 0.0147 | 0.0041 | 0.0009 | 0.0002 | 0 | 0 | 0 |
| 4 | **0.0361** | **0.0399** | **0.0221** | **0.0081** | 0.0022 | 0.0005 | 0.0001 | 0 | 0 | 0 |
| 5 | **0.016** | **0.0176** | **0.0098** | **0.0036** | **0.001** | 0.0002 | 0 | 0 | 0 | 0 |
| 6 | **0.0059** | **0.0065** | **0.0036** | **0.0013** | **0.0004** | **0.0001** | 0 | 0 | 0 | 0 |
| 7 | **0.0019** | **0.0021** | **0.0011** | **0.0004** | **0.0001** | **0** | **0** | 0 | 0 | 0 |
| 8 | **0.0005** | **0.0006** | **0.0003** | **0.0001** | **0** | **0** | **0** | **0** | 0 | 0 |
| 9 | **0.0001** | **0.0001** | **0.0001** | **0** | **0** | **0** | **0** | **0** | **0** | 0 |

# Predicting football games results

Probability of Fiorentina winning: Sum of all the probabilities in the upper triangle, without diagonal

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0.0363 | **0.0401** | **0.0222** | **0.0082** | **0.0023** | **0.0005** | **0.0001** | **0** | **0** | **0** |
| 1 | 0.0803 | 0.0887 | **0.049** | **0.0181** | **0.005** | **0.0011** | **0.0002** | **0** | **0** | **0** |
| 2 | 0.0887 | 0.098 | 0.0542 | **0.02** | **0.0055** | **0.0012** | **0.0002** | **0** | **0** | **0** |
| 3 | 0.0654 | 0.0722 | 0.0399 | 0.0147 | **0.0041** | **0.0009** | **0.0002** | **0** | **0** | **0** |
| 4 | 0.0361 | 0.0399 | 0.0221 | 0.0081 | 0.0022 | **0.0005** | **0.0001** | **0** | **0** | **0** |
| 5 | 0.0160 | 0.0176 | 0.0098 | 0.0036 | 0.001 | 0.0002 | **0** | **0** | **0** | **0** |
| 6 | 0.0059 | 0.0065 | 0.0036 | 0.0013 | 0.0004 | 0.0001 | 0 | **0** | **0** | **0** |
| 7 | 0.0019 | 0.0021 | 0.0011 | 0.0004 | 0.0001 | 0 | 0 | 0 | **0** | **0** |
| 8 | 0.0005 | 0.0006 | 0.0003 | 0.0001 | 0 | 0 | 0 | 0 | 0 | **0** |
| 9 | 0.0001 | 0.0001 | 0.0001 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

# Predicting football games results

Probability of game ending in draw

```
sum(diag(a))
## [1] 0.1963
```

Probability of Lazio winning

```
sum(a[lower.tri(a, diag=F)])
## [1] 0.6239
```

Probability of Fiorentina winning

```
sum(a[upper.tri(a, diag=F)])
## [1] 0.1795
```

# Predicting football games results

- Suppose $X$ is a Poisson random variable with mean $\lambda_x$ that describes the number of the goals the home team scores on average.
- Suppose $Y$ is a Poisson random variable with mean $\lambda_y$ that describes the number of the goals the away team scores on average.
- $Z = X - Y$ random variable has a Skellam distribution and $Z$ represents the outcome of the game with respect to the Home team. If $Z > 0$, then the home team has won, if $Z = 0$, then it was a draw, and if $Z < 0$, the home team has lost.

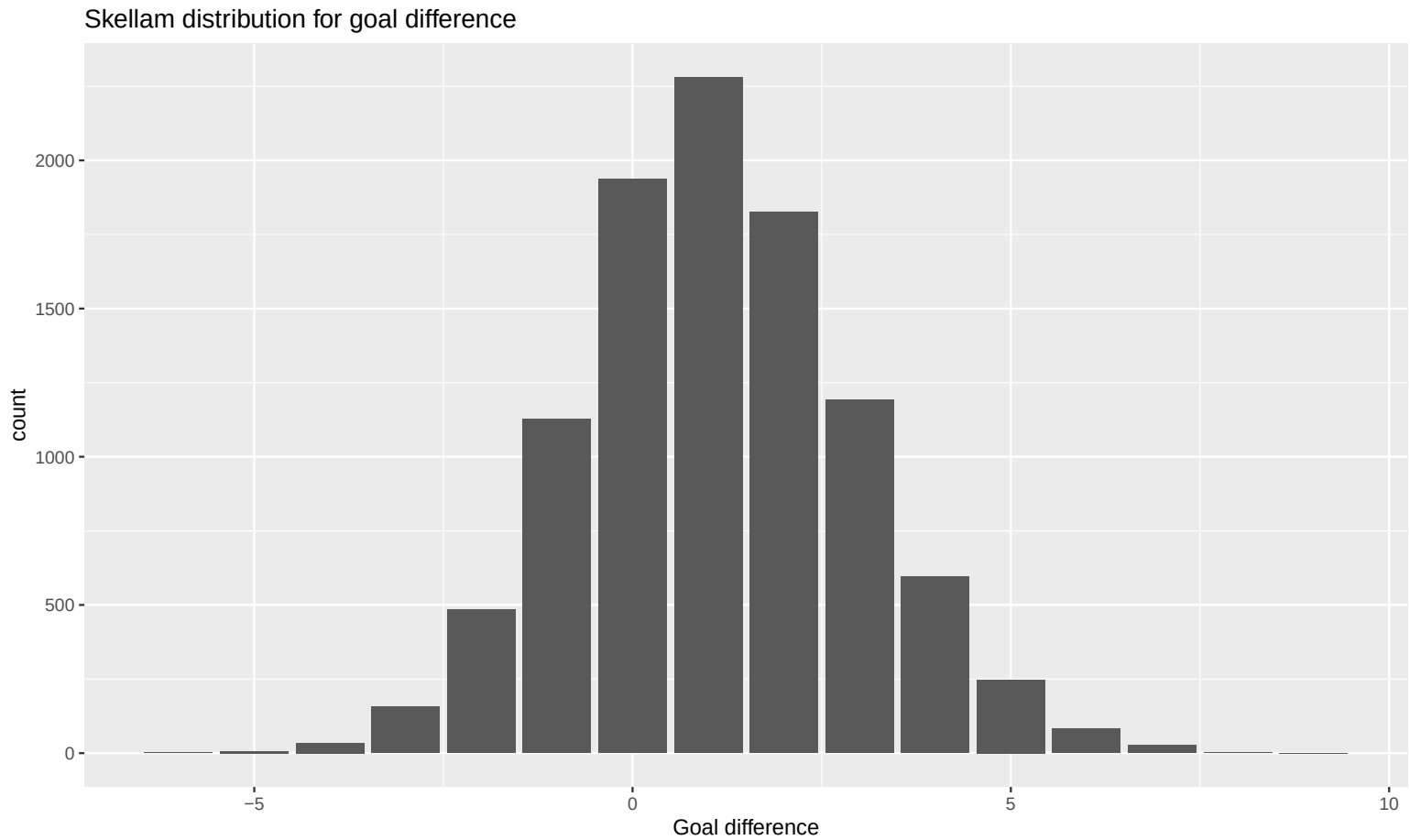
Library for skellam distribution

```
install.packages("VGAM")
library(VGAM)
```

# Predicting football games results

## Histogram for goal difference

```
set.seed(1)
# lazio = 2.21 fiorentina = 1.10
k = rskellam(10000,mu1=lazio, mu2=fiorentina)
ggplot()+geom_bar(aes(x=k))+
  labs(x="Goal difference", title="Skellam distribution for goal difference")
```



# Predicting football games results

Calculating probabilities

Probability of home team to win

```
sum(k>0)/10000
## [1] 0.6253
```

Probability of draw

```
sum(k==0)/10000
## [1] 0.1936
```

Probability of away team to win

```
sum(k<0)/10000
## [1] 0.1811
```

# Predicting with Ratings

# Predicting with Ratings

- Every team can have some kind of rating, describing its ability to win.
- In football (soccer) the rating can be expressed as the number of points earned, number of goals scored, number of goals conceded, etc.
- One way to go with rating football teams is to calculate goal supremacy for each team.

**Goal supremacy = Number of scored goals- Number of conceded goals**

# Predicting with Ratings

- Tottenham playing Leeds United (2002).
- Looking at the goal supremacy (goal difference) for each team for their last 6 games
- Bothe teams will have Goal supremacy rating (we will call it Power Rating) of -3
- What will be your best guess for probability of Tottenham to win? Draw ?

| Match | Tottenham | Leeds |
|---|---|---|
| 1 | Arsenal 3:0 Tottenham | Leeds 2:4 Bolton |
| 2 | Sunderland 2:0 Tottenham | West Ham 3:4 Leeds |
| 3 | Tottenham 0:0 Chelsea | Leeds 0:1 Everton |
| 4 | Liverpool 2:1 Tottenham | Middlesbrough 2:2 Leeds |
| 5 | Tottenham 3:1 Bolton | Leeds 0:1 Liverpool |
| 6 | Blackburn 1:2 Tottenham | Aston Villa 0:0 Leeds |

Adapted from: "Fixed Odds Sports Betting" – Joseph Buchdahl

# Predicting with Ratings

Steps for Power Rating calculation

- Get the Goal Supremacy Rating (Power) for each team $n$ games before the given game.
- Calculate Power Difference for home/away teams – Match rating
- Look at the relative frequencies for home team win, draw and away team win for the given power difference.
- Those will be the probabilities

# Predicting with Ratings

Kind of table we need

| Match rating | No. of home wins | No. of draws | No. of away wins | % of total matches | % of home wins | % of home draws | % of away wins |
|---|---|---|---|---|---|---|---|
| -11 | 75 | 54 | 91 | 1.57% | 34.1% | 24.5% | 41.4% |
| -10 | 84 | 94 | 91 | 1.92% | 31.2% | 34.9% | 33.8% |
| -9 | 123 | 91 | 112 | 2.33% | 37.7% | 27.9% | 34.4% |
| -8 | 171 | 113 | 124 | 2.91% | 41.9% | 27.7% | 30.4% |
| -7 | 190 | 121 | 170 | 3.44% | 39.5% | 25.2% | 35.3% |
| -6 | 242 | 202 | 191 | 4.54% | 38.1% | 31.8% | 30.1% |
| -5 | 279 | 212 | 197 | 4.91% | 40.6% | 30.8% | 28.6% |
| -4 | 293 | 219 | 215 | 5.19% | 40.3% | 30.1% | 29.6% |

# Predicting with Ratings

- Subset the f_data_sm to get only the games for England
- Select columns with team names, date, season indicator and final score

```
data(f_data_sm)
england = f_data_sm %>%
  filter(COUNTRY == "England") %>%
  arrange(DATE) %>%
  select("SEASON","DATE","HOMETEAM",  "AWAYTEAM","FTHG", "FTAG")%>%
  mutate(ID = 1:n())
```

reshape the data into long format

```
england_long = england %>% reshape2::melt(
          id.vars = c("ID","HOMETEAM",  "AWAYTEAM",  "SEASON", "DATE"),
          measure.vars = c("FTHG", "FTAG"), variable.name="H_A") %>%
          arrange(ID)


england_long = england_long[order(england_long$ID),]
rownames(england_long) = NULL
```

# Predicting with Ratings

New dataset

```
head(england_long)
##   ID    HOMETEAM  AWAYTEAM SEASON       DATE  H_A value
## 1  1     Arsenal  Coventry   1994 1993-08-14 FTHG     0
## 2  1     Arsenal  Coventry   1994 1993-08-14 FTAG     3
## 3  2 Aston Villa       QPR   1994 1993-08-14 FTHG     4
## 4  2 Aston Villa       QPR   1994 1993-08-14 FTAG     1
## 5  3     Chelsea Blackburn   1994 1993-08-14 FTHG     1
## 6  3     Chelsea Blackburn   1994 1993-08-14 FTAG     2
```

# Predicting with Ratings

Make changes in H_A variable

```
england_long$TEAM1 = ifelse(england_long$H_A =='FTAG',
                            england_long$AWAYTEAM, england_long$HOMETEAM)

england_long$TEAM2 = ifelse(england_long$H_A =='FTAG',
                            england_long$HOMETEAM, england_long$AWAYTEAM)

england_long = england_long %>%
  select(ID, SEASON, DATE, TEAM1, TEAM2, H_A, value)
```

# Predicting with Ratings

```
head(england_long)
##   ID SEASON       DATE       TEAM1       TEAM2  H_A value
## 1  1   1994 1993-08-14     Arsenal    Coventry FTHG     0
## 2  1   1994 1993-08-14    Coventry     Arsenal FTAG     3
## 3  2   1994 1993-08-14 Aston Villa         QPR FTHG     4
## 4  2   1994 1993-08-14         QPR Aston Villa FTAG     1
## 5  3   1994 1993-08-14     Chelsea   Blackburn FTHG     1
## 6  3   1994 1993-08-14   Blackburn     Chelsea FTAG     2
```

# Predicting with Ratings

- Calculate the scored and conceded goals for the last 6 games per season for each team
- rollapply from zoo and lag from dplyr are used here
- At the end change the class of the object back to dataframe (from tibble that is default for dplyr)

```
england_long1 = england_long %>%
group_by(SEASON, TEAM1) %>%
mutate(CUM_SC = dplyr::lag(rollapply(value,6,sum, fill=NA, align='right'),1)) %>%
ungroup() %>%
group_by(SEASON, TEAM2) %>%
mutate(CUM_C = dplyr::lag(rollapply(value,6,sum, fill=NA, align='right'),1)) %>%
filter(CUM_SC != "NA") %>% as.data.frame()
```

# Predicting with Ratings

Important: Round 7 game Arsenal vs Ipswich.

- In last 6 games Arsenal scored 7 goals and conceded 5
- In last 6 games Ipswich scored 7 and conceded 3

```
head(england_long1)
##   ID SEASON       DATE       TEAM1       TEAM2  H_A value CUM_SC CUM_C
## 1 67   1994 1993-09-11     Arsenal     Ipswich FTHG     4      7     3
## 2 67   1994 1993-09-11     Ipswich     Arsenal FTAG     0      7     5
## 3 68   1994 1993-09-11 Aston Villa    Coventry FTHG     0      9     6
## 4 68   1994 1993-09-11    Coventry Aston Villa FTAG     0     11     5
## 5 69   1994 1993-09-11     Chelsea  Man United FTHG     1      6     3
## 6 69   1994 1993-09-11  Man United     Chelsea FTAG     0     14     6
```

# Predicting with Ratings

Get back to wide format

```
england_wide = reshape(england_long1, direction="wide",
                       timevar = "H_A", idvar = "ID",
                       v.names=c("value", "CUM_SC", "CUM_C"))
```

```
options(width=80)
head(england_wide)
```

```
##    ID SEASON       DATE       TEAM1      TEAM2 value.FTHG CUM_SC.FTHG
## 1  67   1994 1993-09-11     Arsenal    Ipswich          4           7
## 3  68   1994 1993-09-11 Aston Villa   Coventry          0           9
## 5  69   1994 1993-09-11     Chelsea Man United          1           6
## 7  70   1994 1993-09-11    Man City        QPR          3           5
## 9  71   1994 1993-09-11     Norwich  Wimbledon          0          11
## 11 72   1994 1993-09-11      Oldham    Everton          0           5
##    CUM_C.FTHG value.FTAG CUM_SC.FTAG CUM_C.FTAG
## 1           3          0           7          5
## 3           6          0          11          5
## 5           3          0          14          6
## 7          11          0          10          7
## 9           6          1           8          7
## 11          6          1           7          9
```

# Predicting with Ratings

Calculate POWERHOME, POWERAWAY and POWERDIFF and RESULT

```
england_wide$POWERHOME = england_wide$CUM_SC.FTHG-england_wide$CUM_C.FTAG
england_wide$POWERAWAY = england_wide$CUM_SC.FTAG-england_wide$CUM_C.FTHG
england_wide$POWERDIFF = england_wide$POWERHOME-england_wide$POWERAWAY

england_wide$RESULT = ifelse(england_wide$value.FTHG>england_wide$value.FTAG,"HW",
ifelse(england_wide$value.FTHG<england_wide$value.FTAG, "AW", "D"))

england_wide = england_wide %>%
  select(SEASON, DATE, TEAM1, TEAM2, value.FTHG,
                  value.FTAG, POWERHOME, POWERAWAY, POWERDIFF, RESULT)
```

# Predicting with Ratings

Take out remaining NA's

```
options(width=80)
england_wide = england_wide[! is.na(england_wide$POWERDIFF),]
head(england_wide)
##    SEASON       DATE       TEAM1      TEAM2 value.FTHG value.FTAG POWERHOME
## 1    1994 1993-09-11     Arsenal    Ipswich          4          0         2
## 3    1994 1993-09-11 Aston Villa   Coventry          0          0         4
## 5    1994 1993-09-11     Chelsea Man United          1          0         0
## 7    1994 1993-09-11    Man City        QPR          3          0        -2
## 9    1994 1993-09-11     Norwich  Wimbledon          0          1         4
## 11   1994 1993-09-11      Oldham    Everton          0          1        -4
##    POWERAWAY POWERDIFF RESULT
## 1          4        -2     HW
## 3          5        -1      D
## 5         11       -11     HW
## 7         -1        -1     HW
## 9          2         2     AW
## 11         1        -5     AW
```

# Predicting with Ratings

Now let's calculate the game result frequencies by Match rating (POWERDIFF)

```
freqs = dcast(england_wide, POWERDIFF~RESULT)
head(freqs)
##   POWERDIFF AW D HW
## 1       -30  1 0  0
## 2       -26  3 0  0
## 3       -25  2 0  0
## 4       -24  3 0  0
## 5       -23  2 2  0
## 6       -22  9 0  1
```

# Predicting with Ratings

Add relative frequencies

```
freqs = data.frame(freqs, freqs[,2:4]/rowSums(freqs[,2:4]))
head(freqs)
##   POWERDIFF AW D HW AW.1 D.1 HW.1
## 1       -30  1 0  0  1.0 0.0  0.0
## 2       -26  3 0  0  1.0 0.0  0.0
## 3       -25  2 0  0  1.0 0.0  0.0
## 4       -24  3 0  0  1.0 0.0  0.0
## 5       -23  2 2  0  0.5 0.5  0.0
## 6       -22  9 0  1  0.9 0.0  0.1
```

# Predicting with Ratings

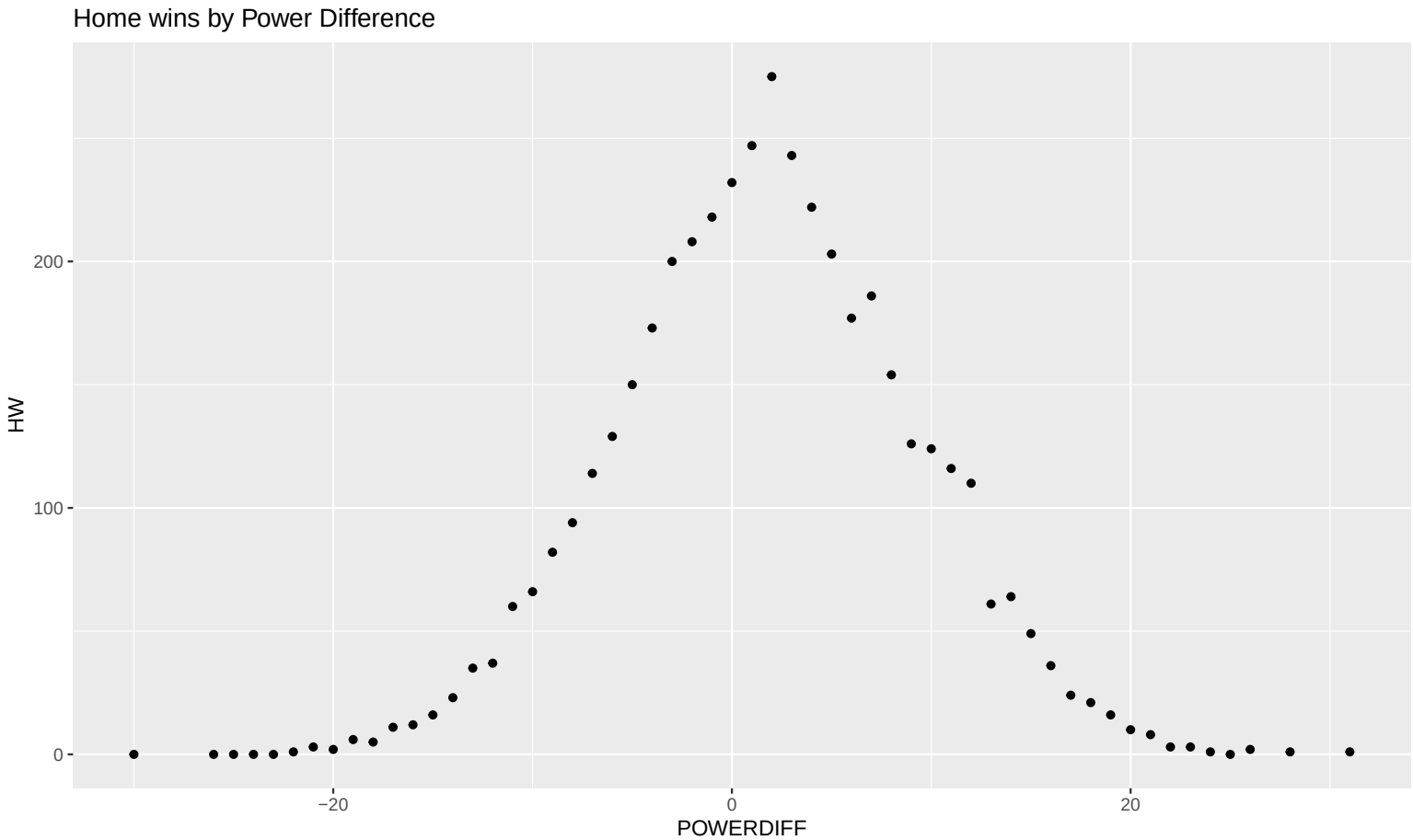
Tottenham vs. Leeds United, two teams with equal Goal Supremacy, the POWERDIFF=0

```
freqs[freqs$POWERDIFF==0,]
##    POWERDIFF  AW   D  HW     AW.1       D.1       HW.1
## 28         0 128 156 232 0.248062 0.3023256 0.4496124
```

# Predicting with Ratings

Distribution of Home Wins by Power difference

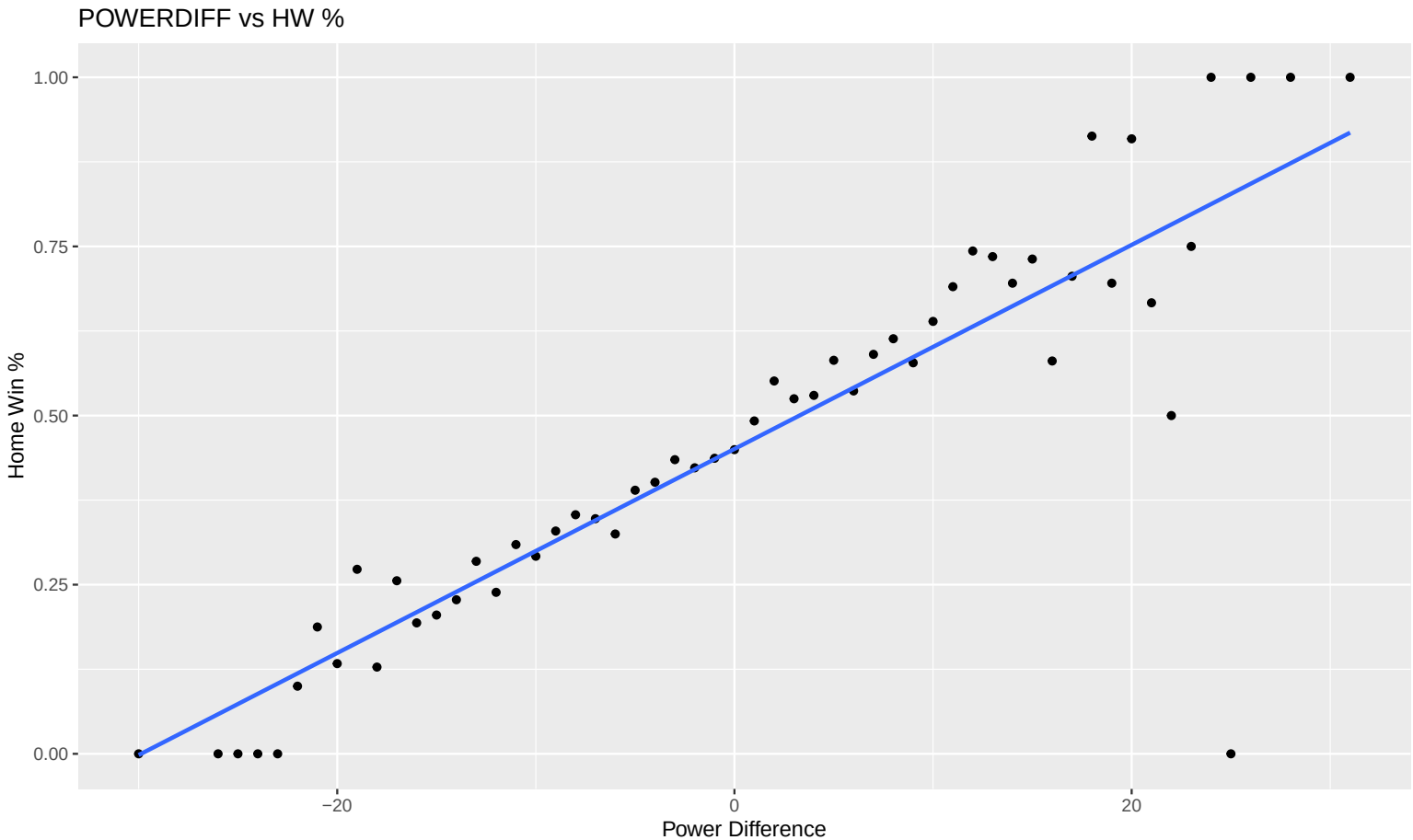
```
ggplot(freqs, aes(x=POWERDIFF, y = HW))+geom_point()+
  ggtitle("Home wins by Power Difference")
```



# Predicting with Ratings

Look at the relationship between probability of home team to win and power difference, linear with exception of some outliers

```
ggplot(freqs, aes(x=POWERDIFF, y=HW.1))+geom_point()+
  geom_smooth(method="lm", se=F)+
  labs(x="Power Difference", y="Home Win %", title = "POWERDIFF vs HW %")
```



# Predicting with Ratings

Look at the model

```
modelH = lm(HW.1~POWERDIFF, data=freqs)
summary(modelH)
##
## Call:
## lm(formula = HW.1 ~ POWERDIFF, data = freqs)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.82766 -0.01895  0.01274  0.05381  0.19095
##
## Coefficients:
##             Estimate Std. Error t value             Pr(>|t|)
## (Intercept) 0.450626   0.018509   24.35 <0.0000000000000002 ***
## POWERDIFF   0.015081   0.001129   13.36 <0.0000000000000002 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1384 on 54 degrees of freedom
## Multiple R-squared:  0.7676, Adjusted R-squared:  0.7633
## F-statistic: 178.4 on 1 and 54 DF,  p-value: < 0.00000000000000022
```

# Predicting with Ratings

Away team probability

```
modelA = lm(AW.1~POWERDIFF, data=freqs)
summary(modelA)
##
## Call:
## lm(formula = AW.1 ~ POWERDIFF, data = freqs)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.18920 -0.07660 -0.04074  0.04295  0.50071
##
## Coefficients:
##             Estimate Std. Error t value             Pr(>|t|)
## (Intercept)  0.33658    0.01688   19.94 <0.0000000000000002 ***
## POWERDIFF   -0.01533    0.00103  -14.89 <0.0000000000000002 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1263 on 54 degrees of freedom
## Multiple R-squared:  0.8041, Adjusted R-squared:  0.8004
## F-statistic: 221.6 on 1 and 54 DF,  p-value: < 0.00000000000000022
```

# Predicting with Ratings

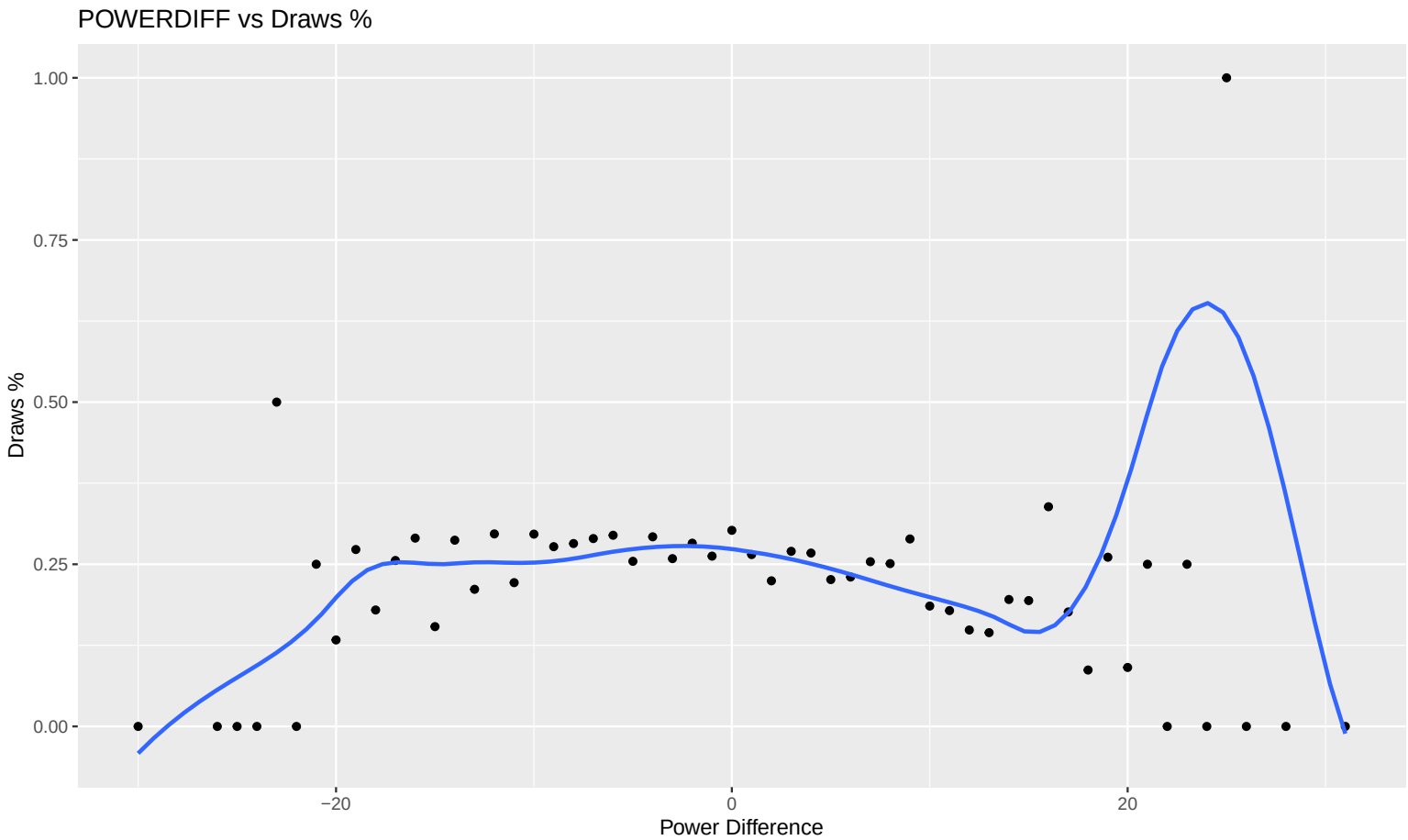
Draw: bad model

```
modelD = lm(D.1~POWERDIFF, data=freqs)
summary(modelD)
##
## Call:
## lm(formula = D.1 ~ POWERDIFF, data = freqs)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.22054 -0.06822  0.03171  0.06748  0.78096
##
## Coefficients:
##             Estimate Std. Error t value         Pr(>|t|)
## (Intercept) 0.2127910  0.0209612  10.152 0.00000000000004 ***
## POWERDIFF   0.0002499  0.0012787   0.195            0.846
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1568 on 54 degrees of freedom
## Multiple R-squared:  0.0007069,  Adjusted R-squared:  -0.0178
## F-statistic: 0.0382 on 1 and 54 DF,  p-value: 0.8458
```

# Predicting with Ratings

Relationship with probability of draw is more like polynomial

```
ggplot(freqs, aes(x=POWERDIFF, y=D.1))+geom_point()+
  geom_smooth(formula=y~poly(x,2), se=F)+
  labs(x="Power Difference", y="Draws %", title = "POWERDIFF vs Draws %")
```



# Predicting with Ratings

Draw: very hard to predict

```
modelD = lm(D.1~poly(POWERDIFF,2), data=freqs)
summary(modelD)
##
## Call:
## lm(formula = D.1 ~ poly(POWERDIFF, 2), data = freqs)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.17669 -0.06951 -0.01010  0.03305  0.85252
##
## Coefficients:
##                     Estimate Std. Error t value           Pr(>|t|)
## (Intercept)          0.21292    0.01978  10.765 0.000000000000006 ***
## poly(POWERDIFF, 2)1  0.03064    0.14801   0.207            0.83678
## poly(POWERDIFF, 2)2 -0.40772    0.14801  -2.755            0.00803 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.148 on 53 degrees of freedom
## Multiple R-squared:  0.1259, Adjusted R-squared:  0.09287
## F-statistic: 3.816 on 2 and 53 DF,  p-value: 0.02831
```

# Predicting with Ratings

Using functions to generalize the procedure and change the country by one line.

```
pick_country <- function(source = f_data_sm, country) {
  country <- source %>%
    filter(COUNTRY == country) %>%
    arrange(DATE) %>%
    select("SEASON","DATE","HOMETEAM",  "AWAYTEAM","FTHG", "FTAG")%>%
    mutate(ID = 1:n())

  country_long <- reshape2::melt(country,
                                 id.vars = c("ID","HOMETEAM",  "AWAYTEAM",  "SEASON", "DATE"),
                                 measure.vars = c("FTHG", "FTAG"), variable.name="H_A")
  country_long <- country_long %>%
    arrange(ID)
  return(list(wide = country,
              long = country_long))
}

prepare_long <- function(data){
  modified <- data %>%
    mutate(TEAM1 = ifelse(H_A == "FTHG", HOMETEAM, AWAYTEAM),
           TEAM2 = ifelse(TEAM1 == HOMETEAM, AWAYTEAM, HOMETEAM)) %>%
    select(c("ID","SEASON","DATE", "TEAM1","TEAM2", "H_A","value"))
  return(modified)
}
```

# Predicting with Ratings

```
goals_scored_conceded <- function(data, n) {
  data_long <- data %>%
    group_by(SEASON, TEAM1) %>%
    mutate(CUM_SC = lag(rollapply(value,n,sum, fill=NA, align='right'),1)) %>%
    ungroup() %>%
    group_by(SEASON, TEAM2) %>%
    mutate(CUM_C = lag(rollapply(value,n,sum, fill=NA, align='right'),1)) %>%
    filter(CUM_SC != "NA")
  data_long <- as.data.frame(data_long)

  data_wide <- reshape(data_long, direction="wide",
                       timevar = "H_A", idvar = "ID",
                       v.names=c("value", "CUM_SC", "CUM_C"))
  return(list(long = data_long,
              wide = data_wide))
}

get_power_diff <- function(data){
  data$POWERHOME <- data$CUM_SC.FTHG-data$CUM_C.FTAG
  data$POWERAWAY <- data$CUM_SC.FTAG-data$CUM_C.FTHG
  data$POWERDIFF <- data$POWERHOME-data$POWERAWAY
  result <- data %>%
    mutate(RESULT = ifelse(value.FTHG > value.FTAG, "HW",
                           ifelse(value.FTHG < value.FTAG, "AW", "D" ) )) %>%
    select(c("SEASON","DATE","TEAM1","TEAM2","value.FTHG",
             "value.FTAG","POWERHOME", "POWERAWAY","POWERDIFF", "RESULT"))
  result <- result[!is.na(result$POWERDIFF),]
  return(result)
}
```

# Predicting with Ratings

```
build_models <- function(freqs) {
  modelH <- lm(HW.1 ~ POWERDIFF, data = freqs)
  modelA <- lm(AW.1 ~ POWERDIFF, data = freqs)
  modelD <- lm(D.1 ~ POWERDIFF, data = freqs)
  modelD_2 <- lm(D.1 ~ poly(POWERDIFF, 2), data = freqs)

  return(list(H = modelH,
              A = modelA,
              D = modelD,
              D_2 = modelD_2))
}

get_final_df_by_country <- function(data, country){
  final_df <- pick_country(source = data, country = country)$wide
  final_df_long <- pick_country(source = data, country = country)$long
  final_df_long <- prepare_long(data = final_df_long)
  final_df_long_1 <- goals_scored_conceded(data = final_df_long, n = 6)$long
  final_df_wide <- goals_scored_conceded(data = final_df_long, n = 6)$wide
  final_df_wide <- get_power_diff(data = final_df_wide)
}
```

# Predicting with Ratings

```
italy_power_ratings <- get_final_df_by_country(f_data_sm, "Italy")
freqs <- dcast(italy_power_ratings, POWERDIFF~RESULT)
head(freqs)
##   POWERDIFF AW D HW
## 1       -25  2 2  0
## 2       -23  2 0  0
## 3       -22  4 1  0
## 4       -21  2 0  0
## 5       -20  6 3  3
## 6       -19  9 4  3
```